Flappy Bird Lab 4 — Prompts

Initial Startup Bug — Fix Before Tasks

Prompt

I have completed the required 10-second video of the original broken Flappy Bird game. Now I need to debug and fix the existing project before implementing the four tasks in its README.

First, inspect the existing code in main.py and the game/ directory. Diagnose why the bird immediately falls to the bottom of the screen and the game ends with "Game over! Final score: 0".

Fix the root cause while preserving the original game structure and appearance. Check the bird's initial position, gravity and flap physics, pipe positions, collision detection, and main game loop. The game should start in a playable state, respond to Space and mouse clicks, and continue until a genuine collision occurs.

Do not implement the other three README features yet. Do not rewrite the entire game or remove collision detection. Explain the exact bug, list the files changed, and provide the steps to test the fix. Then stop and wait for my confirmation before proceeding to Task 1: Refine Collision Detection.

Task 1 — Refine Collision Detection

Prompt

Task 1, Refine Collision Detection, is complete and tested successfully.

Now implement Task 1 from the README: Refine Collision Detection.

Inspect the existing collision detection code in the project, especially game/game_engine.py and game/bird.py. The current implementation checks only the bird's centre point against the pipe rectangles, which can cause collisions to be missed.

Update the collision detection to use the bird's full bounding rectangle and accurately detect overlap with both the upper and lower pipes. Ensure collisions are reliable at different pipe speeds.

Requirements:
1. Preserve the existing game structure and visual appearance.
2. Do not remove gravity, flap physics, or collision detection.
3. Do not implement the Game Over screen, replay/difficulty selection, or sound effects yet.
4. Do not undo the startup bug fix: the game must still wait for the first Space press or mouse click before physics starts.
5. Explain which files you changed and why.

After implementing the change, stop and give me the steps to test it. Do not proceed to the next feature until I confirm that this task works.

Task 2 — Add a Game Over Screen

Prompt

Task 1, Refine Collision Detection, is complete and tested successfully.

Now implement Task 2: Add a Game Over screen to my existing Flappy Bird Pygame project.

Requirements:
1. When the bird hits a pipe or the ground, stop gameplay and display a clear "GAME OVER" message inside the game window.
2. Display the final score on the screen.
3. Keep the existing sky-blue background, green pipes, bird, and overall visual appearance.
4. Preserve the collision detection improvements from Task 1.
5. Preserve the startup fix: the bird must wait for the first Space press or mouse click before physics starts.
6. Stop the bird and pipes from moving after Game Over, but keep the Pygame window responsive.
7. Do not implement replay, difficulty selection, or sound effects yet.
8. Do not rewrite the entire project. Make only the necessary changes and explain which files were modified.

After making the changes, stop and provide testing steps. Wait for my confirmation before implementing the next feature.

Task 3 — Replay and Difficulty Selection

Prompt

Implement Task 3: Replay and Difficulty Selection in my existing Flappy Bird Python/Pygame project.

Before making changes, inspect the existing code, especially game/game_engine.py, main.py, and any other files needed to understand the game loop and input handling.

Requirements:
1. Add a way to replay the game after Game Over without restarting the Python program. Reset the bird position and velocity, pipes, score, spawn timer, game-over state, and any other gameplay state.
2. Add difficulty selection with at least Easy, Medium, and Hard options. Difficulty should affect gameplay, such as pipe speed and/or the gap between pipes.
3. Make the controls and on-screen instructions clear. Allow the player to select a difficulty and start or replay the game using intuitive keyboard or mouse controls.
4. Preserve the existing collision detection improvements, starting behavior, score display, and Game Over screen.
5. Keep the implementation compatible with the project's existing structure. Avoid unnecessary changes to unrelated files or features.
6. Inspect the README and follow its exact Task 3 requirements if they differ from this prompt.

After implementing, explain which files were changed and how to test the feature. Do not implement Task 4 or sound effects yet.

Implementation Notes

Difficulty settings used in the Task 3 implementation:

```
Easy:   speed = 3, gap = 190, interval = 100
Medium: speed = 4, gap = 150, interval = 90
Hard:   speed = 6, gap = 120, interval = 80
```

Controls: Space or mouse click to start; 1/2/3 to select Easy/Medium/Hard; R to replay; Q or Escape to exit.

GitHub History

The updated project and demonstration videos were pushed to https://github.com/WonderAks/Lab-4. Retained commit identifiers: 9a2b2fd (updated code) and f4f0b60 (videos).